Course Compendium: AGE 219 Introduction to Computer Programming

Instructor: Dr Kadeghe Fue, Senior Lecturer

Office: Electronics and Precision Agriculture Lab

Section 1: Introduction to Computer Programming

1.1 Welcome & Course Overview

Welcome to this intensive course on Introduction to Computer Programming using the Python programming language, designed specifically for you, the future engineers from Sokoine University of Agriculture. My name is **Dr Kadeghe Fue,** a Senior Lecturer of Electronics and Digital Agriculture, and I will be your guide through this essential subject. Over the next 12 weeks, we will embark on a journey to explore Python, a compelling and versatile programming language. You will discover how Python can become an indispensable tool in your engineering toolkit, regardless of whether your specialization lies in Agricultural Engineering, Irrigation and Water Resources Engineering, Bioprocessing and Post-harvest Engineering, or perhaps your neighbor's Food Science and Technology, all vital programs offered here at SUA.
It is a common misconception that programming is solely the domain of computer scientists. However, Python's inherent strengths in data analysis, process simulation, automation, and scientific computation make it incredibly relevant for tackling the real-world challenges you will face as engineers. Imagine using Python to analyze crop sensor data to predict yields, model complex water flow systems for efficient irrigation, optimize parameters in a food bioprocessing plant, or automate repetitive data entry tasks. These are just a few examples of how Python empowers modern engineering. This course is structured to equip you with the foundational knowledge and practical skills to leverage Python effectively in your studies and future careers.

Remember that we are using GeeksforGeeks (https://www.geeksforgeeks.org) for all technical explanations. This guide is for reference only. Unfortunately, there are no PPTs for this class, as they have been proven to be failures in delivering this kind of material. You may read the reference: **Rubin, M. J. (2013, March). The effectiveness of live-coding to teach introductory programming. In *Proceedings of the 44th ACM technical symposium on Computer science education* (pp. 651-656).**

Let's begin this exciting journey together!

1.2 Why Python for Agricultural, Irrigation, Bioprocessing, and Food Science Engineers?

The adoption of Python across various engineering disciplines is not accidental. Its design

philosophy emphasizes code readability and simplicity, making it relatively easy to learn and use, even for those new to programming. Furthermore, Python boasts a vast ecosystem of libraries – collections of pre-written code – specifically designed for scientific and engineering tasks. Libraries like NumPy, Pandas, Matplotlib, and SciPy provide powerful tools for numerical computation, data manipulation, visualization, and complex mathematical modeling, significantly reducing development time and effort. This extensive support means you don't have to reinvent the wheel for everyday tasks.

Let's consider some specific applications relevant to the engineering programs here at Sokoine University of Agriculture :

- **Agricultural Engineering:** Python is instrumental in analyzing large datasets from sensors monitoring crop health, soil conditions, and weather patterns. This data can be processed using libraries like Pandas to identify trends, predict yields, or even detect early signs of disease. NumPy enables efficient numerical modeling of soil nutrient dynamics or plant growth. Furthermore, Python scripts can automate tasks related to precision agriculture, such as controlling automated farm equipment or analyzing drone imagery.
- **Irrigation and Water Resources Engineering:** Managing water resources effectively requires sophisticated modeling and data analysis. Python, with libraries like Pandas and NumPy, can process historical rainfall data, analyze streamflow measurements, and model groundwater movement. Matplotlib and Seaborn allow for the visualization of water level data or the results of hydraulic simulations. Python scripts can also help optimize irrigation schedules based on sensor readings and weather forecasts, conserving precious water resources.
- **Bioprocessing and Post-harvest Engineering:** This field heavily relies on experimental data analysis and process modeling. Python and Pandas are ideal for organizing and analyzing data from bioreactors or food processing trials. SciPy offers tools for statistical analysis and optimization, helping engineers determine the best operating conditions for a process. NumPy facilitates the numerical solution of equations describing reaction kinetics or heat transfer during processing and storage. Matplotlib can be used to visualize experimental results and model predictions.
- **Food Science and Technology:** Python helps analyze nutritional composition data, perform statistical quality control checks, and model food processing operations. Libraries like Pandas can manage complex datasets related to ingredients and formulations. NumPy enables calculations for recipe scaling or shelf-life prediction models. Visualization tools like Matplotlib help present quality control data or sensory analysis results.

Beyond these specific applications, Python's clear syntax promotes collaboration, and its large, active community ensures ample resources, tutorials, and support are available online. This makes Python a sustainable and valuable skill for your engineering career.

1.3 Course Objectives & Structure

This course is designed to provide a strong foundation in Python programming and its application in engineering contexts.

Objectives: Upon successful completion of this course, you will be able to:

- Write, execute, and debug basic Python programs independently.
- Effectively utilize fundamental programming constructs such as variables, data types, arithmetic operators, conditional statements (if/elif/else), and loops (for/while).
- Employ core Python data structures, specifically lists and dictionaries, to organize and

manage data.
- Define and use functions to create modular and reusable code.
- Perform basic file input/output operations, including reading from and writing to text files and comma-separated value (CSV) files.
- Leverage the NumPy library for efficient numerical operations using its powerful array objects.
- Utilize the Pandas library to load, inspect, clean, manipulate, and analyze tabular datasets.
- Create informative data visualizations, including line plots, scatter plots, bar charts, and histograms, using Matplotlib and Seaborn.
- Understand the purpose of the SciPy library and its role in solving more advanced scientific and technical computing problems.
- Apply the acquired Python skills to approach and solve simple problems drawn from engineering scenarios relevant to SUA.

Structure: The course follows a logical progression, starting with the fundamentals of the Python language, moving on to essential data structures, and then delving into the core libraries used extensively in scientific and engineering computing. We will conclude with an introduction to more advanced capabilities and a final project. Each module incorporates both lecture sessions to introduce concepts and hands-on lab or practice sessions to reinforce learning through practical application.

1.4 Detailed Course Syllabus

The following table outlines the structure of our journey into Python for engineers:

Module	Topic	Sub-Topics	Lecture (Hrs)	Lab/Practice (Hrs)	Associated Exercises/Assignments
1	The Basics	Syntax (Indentation, Comments), Variables, Data Types (int, float, str, bool), Type Conversion, Input/Output, Arithmetic Operators	2	1	Basic Calculations, I/O Practice
2	Controlling the Flow	Comparison & Logical Operators, if/elif/else, for loops, while loops, range(), break, continue	2	1	Conditional Logic, Loop Tasks
3	Organizing Data	Lists (Creation, Indexing,	2	1	List/Tuple/Dict Manipulations

Module	Topic	Sub-Topics	Lecture (Hrs)	Lab/Practice (Hrs)	Associated Exercises/Assignments
		Slicing, Methods), Tuples (Creation, Immutability), Dictionaries (Key-Value, Operations, Methods)			
4	Building Blocks & File Handling	Functions (def, parameters, return, scope), Docstrings, File I/O (open, with, modes, read/write), Intro to CSV, Importing Modules	2	1	Function Writing, File I/O Tasks
	Assignment 1	**Fundamental Data Processing Task** (Integrates Modules 1-4)	(0.5)	(0.5)	Assignment 1 Due
5	Numerical Computing with NumPy	NumPy Intro, Array Creation, Attributes, Indexing/Slicing, Arithmetic, Basic Ufuncs (sqrt, mean, sum, etc.), Broadcasting Intro	2	2	NumPy Array Operations
6	Data Wrangling with Pandas	Pandas Intro, Series/DataFrame, Reading CSV, Inspection (head, info, etc.), Selection (loc, iloc), Filtering,	2	2	DataFrame Manipulation, Analysis

Module	Topic	Sub-Topics	Lecture (Hrs)	Lab/Practice (Hrs)	Associated Exercises/Assignments
		Missing Data, Grouping			
7	Visualizing Engineering Data	Matplotlib Intro (plot, scatter, bar, hist), Customization (Labels, Title, Legend), Saving Plots, Seaborn Intro, Pandas Plotting	2	2	Creating & Customizing Plots
8	Introduction to SciPy & Course Wrap-up	SciPy Overview, Key Sub-packages (optimize, integrate, etc.), Simple Example (curve_fit or quad), Course Recap	1	1	Apply SciPy Function
	Assignment 2	**Engineering Data Analysis & Visualization Project** (Integrates Modules 5-7)	(0.5)	(0.5)	Assignment 2 Due
Total			**16**	**11**	

(Note: Total time slightly exceeds 25 hours to accommodate assignment introductions/reviews. Adjustments will be made during delivery.)

1.5 Setting Up Your Python Environment

Before we dive into coding, it's essential to set up a working Python environment on your computer. This environment will allow you to write, run, and experiment with Python code throughout the course.

- **Installing Python:** The first step is to install the Python interpreter itself. You can download the latest version directly from the official Python website (www.python.org). Follow the installation instructions specific to your operating system: Windows, macOS, or Linux. During Windows installation, ensure you check the box labeled "Add Python to PATH" to make it easier to run Python from the command line.
- **Anaconda Distribution (Recommended):** For scientific and engineering work, installing the Anaconda distribution is highly recommended. Anaconda bundles Python with many

essential scientific libraries we will use in this course, such as NumPy, Pandas, Matplotlib, and SciPy, along with a helpful package manager (conda). This significantly simplifies the installation process for these libraries. You can download Anaconda from its official website.

- **Code Editors and IDEs:** While you can write Python code in a simple text editor, using a dedicated code editor or an Integrated Development Environment (IDE) will make your life much easier. These tools offer features such as syntax highlighting (making code easier to read), code completion (suggestions as you type), and debugging tools (helping you locate errors). Popular choices include:
    - **Visual Studio Code (VS Code):** A free, powerful, and highly extensible editor.
    - **PyCharm:** A professional IDE with excellent features (a free Community Edition is available).
    - **Spyder:** An IDE often included with Anaconda, popular for scientific computing.
    - **Mu:** A straightforward editor designed for beginners. Choose one that you find comfortable to work with.
- **Jupyter Notebooks:** We will also encounter Jupyter Notebooks, especially when working with data analysis and visualization. Jupyter Notebooks provide an interactive environment where you can combine code, text explanations, mathematical equations, and visualizations in a single document. They are excellent for learning, experimenting, and sharing results. Anaconda typically includes Jupyter Notebook support.

Having a properly configured environment is the first step towards successfully learning and applying Python. Please ensure you have Python (preferably via Anaconda) and a code editor installed before our first lab session. Don't hesitate to ask for assistance if you encounter any issues during setup.

Section 2: Python Programming Fundamentals

This section lays the groundwork for all subsequent Python programming. Mastering these fundamental concepts is crucial for writing effective and correct code.

2.1 Module 1: The Basics (Lecture: 2 hrs, Lab: 1 hr)

This module introduces the absolute essentials of the Python language, focusing on basic syntax, storing data, performing simple calculations, and interacting with the user.

- **Basic Syntax (Indentation, Comments):**
    - **Indentation:** Unlike many other programming languages that use braces {} to define blocks of code, Python uses indentation. Lines of code belonging to the same block (e.g., inside an if statement or a loop) must have the same level of indentation, typically achieved using four spaces per level. Incorrect indentation will lead to errors. This strict indentation enforces a readable code structure.

```
# Correct Indentation
if temperature > 30:
    print("It's hot!")
    print("Turn on the fan.")
else:
    print("It's not too hot.")
```

```
    # Incorrect Indentation (will cause an IndentationError)
    # if temperature > 30:
    # print("It's hot!") # Missing indentation
```

- **Comments:** Comments are explanatory notes within the code that Python ignores during execution. They are vital for making your code understandable to others and to your future self.
  - Single-line comments start with #.
  - Multi-line comments or documentation strings (docstrings) are enclosed in triple quotes ("""Docstring""" or '''Docstring''').

```
# This is a single-line comment explaining the next line
water_level = 5.2 # Sensor reading in meters

"""
This is a multi-line comment (docstring)
explaining a block of code or a function.
"""
print("Processing data...")
```

- **Variables & Assignment:** Variables act as named containers for storing data values. You assign a value to a variable using the assignment operator (=). Python uses dynamic typing, meaning you don't need to declare the type of a variable beforehand; its type is inferred from the assigned value.

```
crop_name = "Maize" # Variable 'crop_name' stores a string
plot_area = 150.75 # Variable 'plot_area' stores a float
num_sensors = 12 # Variable 'num_sensors' stores an integer
is_irrigated = True # Variable 'is_irrigated' stores a boolean

print(crop_name)
print(plot_area)
```

- **Fundamental Data Types (int, float, str, bool):** Python has several built-in data types. The most fundamental are :
  - int: Represents whole numbers (e.g., 10, -5, 0). Used for counting discrete items like livestock or iterations.
  - float: Represents numbers with decimal points (e.g., 3.14, -0.5, 99.9). Used for measurements like temperature, weight, or area.
  - str: Represents sequences of characters (text), enclosed in single (') or double (") quotes (e.g., 'Sokoine', "Agricultural Data"). Used for labels, names, or descriptions.
  - bool: Represents truth values, either True or False. Used for logical conditions and flags.

```
year = 2024         # int
soil_ph = 6.8       # float
sensor_id = "TEMP001" # str
data_valid = True   # bool

print(type(year))      # Output: <class 'int'>
print(type(soil_ph))   # Output: <class 'float'>
```

```
print(type(sensor_id)) # Output: <class 'str'>
print(type(data_valid))# Output: <class 'bool'>
```

- **Type Conversion (int(), float(), str()):** Sometimes you need to convert data from one type to another. Python provides built-in functions for this :
    - int(): Converts to integer (truncates floats, requires strings to be valid whole numbers).
    - float(): Converts to float (adds .0 to integers, requires strings to be valid numbers).
    - str(): Converts to string representation. This is crucial when dealing with user input, which is always read as a string.

```
reading_str = "25.5" # Input from a sensor might be a string
temperature = float(reading_str) # Convert to float for
calculations
print(temperature + 1.0) # Output: 26.5

count_float = 10.8
count_int = int(count_float) # Convert to int (truncates)
print(count_int) # Output: 10

value = 50
value_str = str(value) # Convert int to string for concatenation
print("The value is: " + value_str) # Output: The value is: 50
```

- **Basic Input (input()) & Output (print()):**
    - input(): Prompts the user for input and returns it as a string. Remember to convert the input if you need a number.
    - print(): Displays values or messages to the console. You can print multiple items separated by commas.

```
user_name = input("Enter your name: ")
print("Jambo,", user_name)

radius_str = input("Enter the radius of the field (meters): ")
radius = float(radius_str) # Convert input string to float
area = 3.14159 * radius * radius # Approximate calculation
print("The area of the field is approximately", area, "square
meters.")
```

- **Arithmetic Operators (+, -, *, /, //, %, **):** Python supports standard arithmetic operations essential for engineering calculations :
    - + (Addition), - (Subtraction), * (Multiplication)
    - / (Division): Always results in a float.
    - // (Floor Division): Divides and rounds down to the nearest whole number (integer).
    - % (Modulus): Returns the remainder of a division.
    - ** (Exponentiation): Raises a number to a power.

```
a = 17
b = 5

print(f"{a} + {b} = {a + b}")     # Output: 17 + 5 = 22
```

```
print(f"{a} - {b} = {a - b}")      # Output: 17 - 5 = 12
print(f"{a} * {b} = {a * b}")      # Output: 17 * 5 = 85
print(f"{a} / {b} = {a / b}")      # Output: 17 / 5 = 3.4
print(f"{a} // {b} = {a // b}")    # Output: 17 // 5 = 3
print(f"{a} % {b} = {a % b}")      # Output: 17 % 5 = 2
print(f"{b} ** 2 = {b ** 2}")    # Output: 5 ** 2 = 25
```

- **Exercises:**
  1. **Area Calculator:** Ask the user for the length and width of a rectangular field and calculate its area.
  2. **Temperature Converter:** Ask the user for a temperature in Celsius and convert it to Fahrenheit (F = (C * 9/5) + 32).
  3. **User Greeting:** Ask the user for their name and year of birth, then greet them and calculate their approximate age.
  4. **Number Expansion:** Ask for an integer n and compute n + nn + nnn.

2.2 Module 2: Controlling the Flow (Lecture: 2 hrs, Lab: 1 hr)

Programs often need to make decisions or repeat actions. This module covers the control flow structures that enable this logic.

- **Comparison Operators (==, !=, >, <, >=, <=):** These operators compare two values and return a Boolean (True or False) result, forming the basis of decision-making.

```
soil_moisture = 0.4 # Example value (e.g., 40%)
threshold = 0.3

print(f"Is moisture below threshold? {soil_moisture < threshold}")
# False
print(f"Is moisture equal to 0.4? {soil_moisture == 0.4}")    #
True
print(f"Is moisture not equal to threshold? {soil_moisture!=
threshold}") # True
```

- **Logical Operators (and, or, not):** These operators combine multiple Boolean conditions :
  - and: True only if *both* conditions are True.
  - or: True if *at least one* condition is True.
  - not: Reverses the Boolean value (True becomes False, False becomes True).

```
temperature = 25.0
humidity = 60.0

# Check if conditions are suitable for a specific crop
is_suitable = temperature > 20 and humidity < 70
print(f"Conditions suitable? {is_suitable}") # True

# Check if irrigation is needed (low moisture OR high temp)
needs_irrigation = soil_moisture < 0.2 or temperature > 35
print(f"Needs irrigation? {needs_irrigation}") # False (based on
previous soil_moisture)
```

```
# Using not
is_raining = False
can_work_outside = not is_raining
print(f"Can work outside? {can_work_outside}") # True
```

- **Conditional Statements (if, elif, else):** These statements allow your program to execute different blocks of code based on whether conditions are true or false.
    - if: Executes a block if its condition is True.
    - elif (else if): Checked only if the preceding if (or elif) was False. Allows checking multiple alternative conditions.
    - else: Executes if *none* of the preceding if or elif conditions were True. It's optional. Indentation defines which statements belong to each block.

```
ph_level = 5.8

if ph_level < 5.5:
    print("Soil is too acidic. Add lime.")
elif ph_level > 7.5:
    print("Soil is too alkaline. Add sulfur.")
else: # ph_level is between 5.5 and 7.5 (inclusive)
    print("Soil pH is within the acceptable range.")
```

- **Loops (for, while):** Loops are used to repeat a block of code multiple times.
    - **for loop:** Iterates over a sequence (like a list, tuple, string, or range) one item at a time.

```
# Iterate over a list of sensor readings
sensor_readings = [22.1, 22.5, 23.0, 22.8, 22.6]
total_temp = 0.0
for temp in sensor_readings:
    print(f"Processing reading: {temp}")
    total_temp += temp
average_temp = total_temp / len(sensor_readings)
print(f"Average temperature: {average_temp:.2f}")

# Iterate a specific number of times using range()
# range(n) generates numbers from 0 up to (but not including)
n
print("Simulating 5 time steps:")
for i in range(5):
    print(f"Time step {i+1}")
```

    The range() function is very useful for numerical loops. range(start, stop, step) allows more control.
    - **while loop:** Repeats a block as long as a condition remains True. It's important to ensure the condition eventually becomes False to avoid an infinite loop.

```
# Simulate filling a water tank until it reaches a certain
level
current_level = 0.0 # meters
target_level = 2.0 # meters
```

```
    fill_rate = 0.1 # meters per iteration

    while current_level < target_level:
        print(f"Current water level: {current_level:.1f} m")
        current_level += fill_rate
        # In a real simulation, add a time delay or check sensor

    print(f"Target level reached: {current_level:.1f} m")
```

- **Loop Control (break, continue):** These statements modify the standard behavior of loops :
    - break: Immediately exits the innermost for or while loop it's contained within.
    - continue: Skips the rest of the current iteration and proceeds to the next iteration of the loop.

```
# Find the first temperature reading above 25 degrees
temps =
[span_0](start_span)[span_0](end_span)[span_1](start_span)[span_1]
(end_span)[span_2](start_span)[span_2](end_span)[span_3](start_spa
n)[span_3](end_span)[span_4](start_span)[span_4](end_span)[span_5]
(start_span)[span_5](end_span)
found_temp = None
for temp in temps:
    if temp > 25:
        found_temp = temp
        print(f"First high temperature found: {found_temp}")
        break # Exit the loop once found
if found_temp is None:
    print("No high temperatures found.")

# Process only positive sensor readings
readings = [10, -5, 15, 0, -2, 20]
print("Processing positive readings:")
for reading in readings:
    if reading <= 0:
        continue # Skip negative or zero readings
    print(f"Processing valid reading: {reading}")
```

- **Exercises:**
    1. **Grade Classifier:** Ask the user for a numerical score and print the corresponding grade (e.g., A, B, C based on ranges).
    2. **Sum of Evens:** Calculate the sum of all even numbers between 1 and 50 using a for loop and range().
    3. **Input Validation:** Use a while loop to repeatedly ask the user for a positive number until they enter one.
    4. **Prime Number Checker:** Ask the user for a number and use a loop and break to determine if it's a prime number.
    5. **List Search:** Given a list of items, ask the user for an item to search for. Use a loop to check if the item is in the list and print the result.

2.3 Module 3: Organizing Data (Lecture: 2 hrs, Lab: 1 hr)

Python offers versatile built-in data structures to store and organize collections of data efficiently. This module focuses on lists, tuples, and dictionaries.

- **Lists:** Lists are ordered, mutable (changeable) sequences of items, enclosed in square brackets ``. They can hold items of different data types.
    - **Creation:** my_list =
    - **Indexing:** Access elements by position (0-based). my_list returns 1. my_list[-1] returns True.
    - **Slicing:** Extract sub-lists. my_list[1:3] returns ['apple', 3.14].
    - **Mutability:** You can change elements: my_list[span_8](start_span)[span_8](end_span) = "banana".
    - **Common Methods:**
        - append(item): Adds an item to the end. my_list.append(5)
        - remove(item): Removes the first occurrence of an item. my_list.remove(3.14)
        - pop(index): Removes and returns the item at index (or the last item if index is omitted). popped_item = my_list.pop()
        - sort(): Sorts the list in place (if items are comparable). num_list = [span_13](start_span)[span_13](end_span)[span_14](start_span)[span_14](end_span)[span_15](start_span)[span_15](end_span); num_list.sort() results in [span_16](start_span)[span_16](end_span)[span_17](start_span)[span_17](end_span)[span_18](start_span)[span_18](end_span).
        - len(list): Returns the number of items in the list.

```
# Storing daily rainfall measurements (in mm)
rainfall_data = [5.2, 0.0, 12.1, 1.5, 0.0]

# Accessing data
print(f"Day 1 rainfall: {rainfall_data} mm")

# Adding a new day's data
rainfall_data.append(8.3)
print(f"Updated data: {rainfall_data}")

# Removing the second day's data (index 1)
rainfall_data.pop(1)
print(f"Data after removing Day 2: {rainfall_data}")

# Sorting the data
rainfall_data.sort()
print(f"Sorted rainfall data: {rainfall_data}")

# Finding the number of days recorded
print(f"Number of days: {len(rainfall_data)}")
```

- **Tuples:** Tuples are ordered, immutable (unchangeable) sequences of items, enclosed in parentheses (). Once created, their elements cannot be altered. They are often used for

fixed collections of items, like coordinates or database records.

- **Creation:** my_tuple = (10, 20, "stable"). Note: a single-element tuple needs a trailing comma: single = (5,).
- **Indexing & Slicing:** Works the same way as lists. my_tuple returns 10. my_tuple[1:] returns (20, "stable").
- **Immutability:** You *cannot* change elements: my_tuple = 5 would raise a TypeError. This immutability makes them suitable for use as dictionary keys if all elements are immutable.

```
# Representing GPS coordinates (latitude, longitude)
location_sua = ( -6.8514, 37.6572 )

print(f"Latitude: {location_sua}")
print(f"Longitude:
{location_sua[span_9](start_span)[span_9](end_span)}")

# Attempting to change the tuple will fail
# location_sua = -7.0 # Raises TypeError

# Tuples can be used as dictionary keys
coordinate_data = { location_sua: "Main Campus", (-6.853, 37.66):
"Admin Building" }
print(coordinate_data[location_sua]) # Output: Main Campus
```

- **Dictionaries:** Dictionaries store data as key-value pairs, enclosed in curly braces {}. Keys must be unique and immutable (strings, numbers, tuples), while values can be anything. Dictionaries are optimized for retrieving values when you know the key. As of Python 3.7, dictionaries maintain insertion order.
  - **Creation:** experiment_params = {"temperature": 25.5, "pressure": 1.0, "catalyst": "Zeolite"}
  - **Accessing:** Use the key in square brackets or the `.get()` method. `experiment_params["temperature"]` returns `25.5`. `experiment_params.get("humidity", "N/A")` returns `"N/A"` if "humidity" key doesn't exist. Using with a non-existent key raises a KeyError.
  - **Adding/Modifying:** Assign a value to a key. If the key exists, it updates the value; otherwise, it adds a new key-value pair. experiment_params["duration"] = 120 (adds). experiment_params["temperature"] = 26.0 (modifies).
  - **Removing:** Use pop(key) to remove a key and get its value, or del dictionary[key]. catalyst_used = experiment_params.pop("catalyst"). del experiment_params["pressure"].
  - **Common Methods:**
    - keys(): Returns a view of all keys.
    - values(): Returns a view of all values.
    - items(): Returns a view of all key-value pairs (as tuples).

```
# Storing crop yield data (Crop Name: Yield in tons/hectare)
crop_yields = {
    "Maize": 4.5,
    "Beans": 1.8,
    "Rice": 3.2
```

```
}

# Accessing yield for Maize
print(f"Maize yield: {crop_yields['Maize']} tons/ha")

# Adding yield for Sunflower
crop_yields = 2.1
print(f"Added Sunflower: {crop_yields}")

# Updating Rice yield
crop_yields = 3.5
print(f"Updated Rice yield: {crop_yields}")

# Iterating through items
print("\nCrop Yield Report:")
for crop, yield_val in crop_yields.items():
    print(f"- {crop}: {yield_val} tons/ha")

# Getting just the crop names (keys)
print(f"\nCrops recorded: {list(crop_yields.keys())}")
```

Understanding the differences between these structures is key. Lists are for ordered collections that might change. Tuples are for ordered collections that should not change. Dictionaries are for associating unique keys with values for fast lookups.

- **Exercises:**
    1. **Sensor Data:** Store a sequence of 10 temperature readings in a list. Calculate the average temperature. Add a new reading. Remove the oldest reading.
    2. **Experiment Log:** Create a dictionary to store parameters for an experiment (e.g., 'sample_id', 'temperature', 'duration'). Print the parameters. Add a new parameter 'result'.
    3. **Field Coordinates:** Store the coordinates of the four corners of a rectangular field as tuples within a list. Access and print the coordinates of the third corner.
    4. **Word Frequency:** Read a sentence and use a dictionary to count the frequency of each word.

2.4 Module 4: Building Blocks & File Handling (Lecture: 2 hrs, Lab: 1 hr)

This module introduces functions for creating reusable code blocks and file input/output (I/O) for interacting with external data.

- **Defining Functions (def, parameters, return):** Functions allow you to group a block of code, give it a name, and execute it on demand. This promotes code reuse, organization, and modularity.
    - Use the def keyword, followed by the function name, parentheses (), and a colon :.
    - Parameters are variables listed inside the parentheses, acting as inputs to the function.
    - The indented code block below def is the function body.
    - The return statement sends a value back from the function. If omitted, the function

returns None.

```
import math # Import the math module for pi

def calculate_circle_area(radius):
    """Calculates the area of a circle given its radius.""" #
Docstring
    if radius < 0:
        return None # Indicate invalid input
    area = math.pi * radius ** 2
    return area # Return the calculated area

# Calling the function
field_radius = 10.0
calculated_area = calculate_circle_area(field_radius)

if calculated_area is not None:
    print(f"The area of a circle with radius {field_radius} is
{calculated_area:.2f}")
else:
    print("Invalid radius provided.")
```

- **Function Arguments (Required, Default):**
  - **Required Arguments:** Must be provided when calling the function, matching the parameters in order.
  - **Default Arguments:** Assigned a default value in the function definition using =. If the caller doesn't provide a value for that argument, the default is used.

```
def calculate_volume(length, width, height=1.0):
    """Calculates volume, assuming height is 1.0 if not
provided."""
    return length * width * height

volume1 = calculate_volume(5, 3, 2) # All arguments provided
print(f"Volume 1: {volume1}") # Output: 30

volume2 = calculate_volume(5, 3) # Height uses default value 1.0
print(f"Volume 2: {volume2}") # Output: 15
```

- **Variable Scope (Local vs. Global):**
  - **Local Scope:** Variables defined *inside* a function are local to that function and cannot be accessed from outside.
  - **Global Scope:** Variables defined *outside* any function are global and can be accessed (read) from anywhere. To modify a global variable inside a function, use the global keyword. It's generally good practice to minimize the use of global variables.

```
gravity = 9.81 # Global variable

def calculate_potential_energy(mass, height):
    local_energy = mass * gravity * height # Uses global
```

```
'gravity', defines local 'local_energy'
    print(f"Inside function: Energy = {local_energy}")
    return local_energy

pe = calculate_potential_energy(10, 5)
print(f"Outside function: Potential Energy = {pe}")
# print(local_energy) # This would cause an error (NameError)
```

- **Docstrings:** Documentation strings (docstrings) are triple-quoted strings placed immediately after the def line. They explain what the function does, its parameters, and what it returns. They are crucial for code documentation and can be accessed using help(function_name) or function_name.__doc__. Always include docstrings for your functions. (See calculate_circle_area example above).
- **Basic File I/O (open, modes 'r', 'w', 'a', read/write methods, with open):** File I/O allows programs to read data from files and write results to files.
    - open(filename, mode): Opens a file and returns a file object.
    - **Modes:** 'r' (read, default), 'w' (write, truncates existing file or creates new), 'a' (append, adds to end or creates new). Add '+' for read/write capabilities (e.g., 'r+'). Add 'b' for binary mode (e.g., 'rb').
    - **Reading Methods:** .read() (entire file), .readline() (one line), .readlines() (all lines into a list).
    - **Writing Method:** .write(string).
    - **Closing:** Crucial to close files (file.close()) to save changes and free resources.
    - **with open(...) as file::** The recommended way to handle files. It automatically closes the file when the block is exited, even if errors occur.

```
# Writing data to a file
sensor_data =
try:
    with open("sensor_log.csv", "w") as outfile: # 'w' mode
creates/overwrites
        for line in sensor_data:
            outfile.write(line + "\n") # Add newline character
    print("Data successfully written to sensor_log.csv")
except IOError:
    print("Error: Could not write to file.")

# Reading data from the file
try:
    with open("sensor_log.csv", "r") as infile: # 'r' mode reads
        print("\nReading data from sensor_log.csv:")
        # content = infile.read() # Read entire file
        # print(content)
        lines = infile.readlines() # Read all lines into a list
        for line in lines:
            print(line.strip()) #.strip() removes leading/trailing
whitespace/newlines
except FileNotFoundError:
    print("Error: sensor_log.csv not found.")
```

```
except IOError:
    print("Error: Could not read from file.")
```

- **Introduction to CSV Files:** Comma-Separated Values (CSV) is a common plain text format for storing tabular data (like spreadsheets). Each line represents a row, and values within a row are separated by commas. While Pandas (covered later) is the best tool for complex CSV handling, you can perform basic reading/writing using file I/O and string methods like .split(','). Python also has a built-in csv module for more robust handling.

```
# Basic CSV reading (without csv module)
try:
    with open("sensor_log.csv", "r") as csvfile:
        header = csvfile.readline().strip().split(',') # Read
header
        print(f"\nCSV Header: {header}")
        print("CSV Data:")
        for line in csvfile:
            values = line.strip().split(',')
            if len(values) == len(header): # Basic check
                timestamp = values
                temperature =
float(values[span_10](start_span)[span_10](end_span)) # Convert to
float
                humidity =
float(values[span_23](start_span)[span_23](end_span))    # Convert
to float
                print(f"  Time: {timestamp}, Temp: {temperature},
Humidity: {humidity}")
except FileNotFoundError:
    print("Error: sensor_log.csv not found.")
except (IOError, ValueError, IndexError) as e:
    print(f"Error processing CSV file: {e}")
```

- **Importing Modules (import math):** Python's power comes from its modules. You use the import statement to bring code from a module into your current script. For example, import math gives you access to mathematical functions like math.pi and math.sqrt().

```
import math

radius = 5
circumference = 2 * math.pi * radius
print(f"Circumference: {circumference}")
```

- **Exercises:**
    1. **Function Practice:** Write a function that takes temperature in Fahrenheit and returns it in Celsius.
    2. **File Writer:** Write a function that takes a list of strings and a filename, and writes each string to a new line in the specified file.
    3. **File Reader:** Write a function that takes a filename and reads the entire file content, returning it as a single string.

4. **Simple CSV Reader:** Read the sensor_log.csv file created earlier and print only the temperature values.

Assignment 1: Fundamental Data Processing Task

Description: This assignment integrates the concepts learned in Modules 1 through 4. You will write a Python script that performs the following tasks:

1. **Create a Function:** Define a function analyze_rainfall(input_filename, output_filename, rain_threshold) that takes three arguments: the name of an input file, the name of an output file, and a rainfall threshold (a float).
2. **Read Input File:** The function should read data from the input_filename. Assume the input file is a simple text file where each line contains a single floating-point number representing daily rainfall in millimeters (e.g., rainfall_data.txt). Handle potential FileNotFoundError and ValueError (if a line doesn't contain a valid number).

```
# Example rainfall_data.txt
5.2
0.0
12.1
1.5
0.0
25.8
3.5
invalid_data
8.0
```

3. **Process Data:** Inside the function, calculate the following:
    - Total number of valid days recorded.
    - Total rainfall recorded.
    - Average rainfall per recorded day.
    - Number of days where rainfall exceeded the rain_threshold.
4. **Write Output File:** The function should write a summary report to the output_filename. The report should clearly state the calculated statistics. Handle potential IOError during writing.

```
# Example output_report.txt
Rainfall Analysis Report
------------------------
Input File: rainfall_data.txt
Total Valid Days Recorded: 7
Total Rainfall: 56.1 mm
Average Daily Rainfall: 8.01 mm
Days with Rainfall > 10.0 mm: 2
```

5. **Main Script:** In the main part of your script (outside the function), call the analyze_rainfall function with appropriate filenames (e.g., "rainfall_data.txt", "output_report.txt") and a threshold (e.g., 10.0).

Relevance: This task mirrors common engineering scenarios involving reading sensor or experimental data from files, performing basic calculations and statistical summaries, applying conditional logic (threshold check), handling potential errors, and generating a report file. It

reinforces file I/O, functions, loops, conditionals, basic data types, and error handling.

Section 3: Python Libraries for Engineering Applications

While Python's built-in features are powerful, specialized libraries unlock its true potential for engineering tasks. This section introduces the core libraries for numerical computing, data analysis, and visualization.

3.1 Module 5: Numerical Computing with NumPy (Lecture: 2 hrs, Lab: 2 hrs)

NumPy (Numerical Python) is the fundamental package for scientific computing in Python. It provides a powerful N-dimensional array object, sophisticated functions, and tools for integrating C/C++ and Fortran code. For engineers dealing with numerical data, simulations, or mathematical modeling, NumPy is indispensable.

- **Introduction to NumPy:** Why use NumPy? Standard Python lists are flexible but inefficient for large numerical operations. NumPy arrays (ndarray) are densely packed arrays of homogeneous type. Mathematical operations on these arrays are executed by optimized, pre-compiled C code, leading to significant performance improvements, especially for large datasets. NumPy forms the foundation upon which many other scientific libraries, including Pandas and SciPy, are built. To use NumPy, you typically import it with the alias np: import numpy as np.
- **Creating NumPy Arrays:** There are several ways to create NumPy arrays :
    - np.array(list_or_tuple): Creates an array from a Python list or tuple. The data type is inferred unless specified with dtype.
    - np.arange(start, stop, step): Creates an array with evenly spaced values within a given range (similar to Python's range but returns a NumPy array).
    - np.zeros(shape): Creates an array filled with zeros. shape is a tuple (e.g., (3, 4) for a 3x4 array).
    - np.ones(shape): Creates an array filled with ones.
    - np.linspace(start, stop, num): Creates an array with num evenly spaced values between start and stop (inclusive).

```
import numpy as np

# From list
list_data =
[[span_28](start_span)[span_28](end_span)[span_32](start_span)[spa
n_32](end_span)[span_36](start_span)[span_36](end_span),
[span_40](start_span)[span_40](end_span)[span_44](start_span)[span
_44](end_span)[span_48](start_span)[span_48](end_span)]
arr_a = np.array(list_data)
print("Array from list:\n", arr_a)

# Using arange
arr_b = np.arange(0, 10, 2) # Start=0, Stop=10 (exclusive), Step=2
```

```
print("\nArray using arange:", arr_b)

# Using zeros
arr_c = np.zeros((2, 3)) # 2 rows, 3 columns
print("\nArray of zeros:\n", arr_c)

# Using ones
arr_d = np.ones(4) # 1D array of 4 ones
print("\nArray of ones:", arr_d)

# Using linspace
arr_e = np.linspace(0, 1, 5) # 5 points from 0 to 1 inclusive
print("\nArray using linspace:", arr_e)
```

- **Array Attributes:** NumPy arrays have useful attributes to understand their structure :
  - .shape: A tuple representing the dimensions (rows, columns, etc.).
  - .dtype: The data type of the elements (e.g., int64, float64).
  - .ndim: The number of dimensions (axes).
  - .size: The total number of elements.

```
print(f"\nAttributes of arr_a:")
print(f"  Shape: {arr_a.shape}") # Output: (2, 3)
print(f"  Data Type: {arr_a.dtype}") # Output: int64 (depends on
system)
print(f"  Dimensions: {arr_a.ndim}") # Output: 2
print(f"  Size: {arr_a.size}") # Output: 6
```

- **Indexing & Slicing:** Accessing elements and sub-arrays works similarly to lists but extends to multiple dimensions.
  - **1D:** arr[index], arr[start:stop:step].
  - **2D:** arr[row_index, col_index], arr[row_start:row_stop, col_start:col_stop]. Colons : can be used to select entire rows or columns.

```
arr_2d =
np.array([[span_52](start_span)[span_52](end_span)[span_53](start_
span)[span_53](end_span)[span_54](start_span)[span_54](end_span),
[span_55](start_span)[span_55](end_span)[span_56](start_span)[span
_56](end_span)[span_57](start_span)[span_57](end_span),
[span_58](start_span)[span_58](end_span)[span_59](start_span)[span
_59](end_span)[span_60](start_span)[span_60](end_span)])
print("\n2D Array:\n", arr_2d)

# Single element (row 1, column 2)
print(f"\nElement at
[span_61](start_span)[span_61](end_span)[span_63](start_span)[span
_63](end_span):
{arr_2d[span_62](start_span)[span_62](end_span)[span_64](start_spa
n)[span_64](end_span)}") # Output: 22

# Slice (rows 0 and 1, column 1 onwards)
```

```
print("\nSlice [0:2, 1:]:\n", arr_2d[0:2, 1:])
# Output:
# [[11 12]
#  [21 22]]

# Select a single row (row 0)
print("\nRow 0:", arr_2d[0, :]) # Output: [10 11 12]

# Select a single column (column 1)
print("\nColumn 1:", arr_2d[:, 1]) # Output: [11 21 31]
```

- **Element-wise Arithmetic:** Standard arithmetic operators (+, -, *, /, **) operate element by element on NumPy arrays of the same shape.

```
arr1 =
np.array([span_29](start_span)[span_29](end_span)[span_33](start_s
pan)[span_33](end_span)[span_37](start_span)[span_37](end_span))
arr2 =
np.array([span_41](start_span)[span_41](end_span)[span_45](start_s
pan)[span_45](end_span)[span_49](start_span)[span_49](end_span))
ones_arr = np.ones(3)

print("\nElement-wise operations:")
print(f"arr1 + arr2 = {arr1 + arr2}") # Output: [5 7 9]
print(f"arr1 * 3 = {arr1 * 3}")     # Output: [3 6 9]
print(f"arr2 ** 2 = {arr2 ** 2}")   # Output: [16 25 36]
print(f"arr1 / ones_arr = {arr1 / ones_arr}") # Output: [1. 2. 3.]
```

- **Basic Universal Functions (ufuncs):** Ufuncs are functions that operate element-wise on ndarray objects, providing fast, vectorized operations. Common examples include:
    - Mathematical: np.sqrt(), np.sin(), np.cos(), np.exp(), np.log().
    - Statistical: np.mean(), np.sum(), np.std() (standard deviation), np.min(), np.max(). These can often operate along specific axes (0 for columns, 1 for rows in 2D arrays).

```
data = np.array([1.0, 4.0, 9.0, 16.0])
print("\nUfuncs examples:")
print(f"Square roots: {np.sqrt(data)}") # Output: [1. 2. 3. 4.]

angles = np.array([0, np.pi/2, np.pi])
print(f"Sines: {np.sin(angles)}") # Output: [0. 1. 1.22e-16]
(approx)

matrix =
np.array([[span_30](start_span)[span_30](end_span)[span_34](start_
span)[span_34](end_span)[span_38](start_span)[span_38](end_span),
[span_42](start_span)[span_42](end_span)[span_46](start_span)[span
_46](end_span)[span_50](start_span)[span_50](end_span)])
print(f"\nMatrix:\n{matrix}")
print(f"Overall mean: {np.mean(matrix)}") # Output: 3.5
```

```
print(f"Mean per column (axis=0): {np.mean(matrix, axis=0)}") #
Output: [2.5 3.5 4.5]
print(f"Sum per row (axis=1): {np.sum(matrix, axis=1)}") # Output:
[ 6 15]
print(f"Standard deviation: {np.std(matrix)}")
```

- **Broadcasting (Briefly):** NumPy can perform operations on arrays of different shapes if certain compatibility rules are met (e.g., operating between a 2D array and a 1D array). This is called broadcasting and avoids the need to explicitly duplicate the smaller array.

```
matrix =
np.array([[span_31](start_span)[span_31](end_span)[span_35](start_
span)[span_35](end_span)[span_39](start_span)[span_39](end_span),
[span_43](start_span)[span_43](end_span)[span_47](start_span)[span
_47](end_span)[span_51](start_span)[span_51](end_span)])
vector =
np.array([span_65](start_span)[span_65](end_span)[span_66](start_s
pan)[span_66](end_span)[span_67](start_span)[span_67](end_span))

# Add vector to each row of matrix via broadcasting
result = matrix + vector
print("\nBroadcasting example (matrix + vector):\n", result)
# Output:
# [[11 22 33]
#  [14 25 36]]
```

- **Exercises:**
    1. **Array Creation:** Create a 1D NumPy array of numbers from 10 to 20. Create a 3x3 NumPy array filled with the value 5.0. Create an array of 10 evenly spaced points between 0 and 1.
    2. **Attributes:** Create a 4x2 NumPy array of random integers. Print its shape, data type, dimensions, and size.
    3. **Slicing:** Create a 5x5 NumPy array with values from 1 to 25. Extract the 3x3 inner matrix. Extract the last column. Extract every second row.
    4. **Calculations:** Create two 1D arrays of the same size. Perform addition, multiplication, and calculate the square root of the first array.
    5. **Statistics:** Create a 2D array. Calculate the sum of all elements, the mean of each row, and the standard deviation of each column..

3.2 Module 6: Data Wrangling with Pandas (Lecture: 2 hrs, Lab: 2 hrs)

Pandas is the cornerstone library for data manipulation and analysis in Python. It provides high-performance, easy-to-use data structures (Series and DataFrame) and data analysis tools, making it essential for working with tabular data like spreadsheets or database tables.

- **Introduction to Pandas:** Pandas excels at handling structured (labeled) data. It simplifies tasks like reading data from various sources, cleaning messy data, selecting subsets, merging datasets, and performing complex aggregations. It integrates seamlessly with NumPy and Matplotlib. The conventional import alias is pd: import pandas as pd.
- **Series & DataFrame Objects:**

- **Series:** A one-dimensional labeled array, like a single column in a spreadsheet, with an index.
- **DataFrame:** A two-dimensional labeled data structure with columns of potentially different types, similar to a spreadsheet or SQL table. It has both row and column indices. DataFrames are the primary objects used in Pandas.

```
import pandas as pd
import numpy as np

# Creating a Series
temps = pd.Series([22.1, 22.5, 23.0, 22.8], name="Temperature")
print("Pandas Series:\n", temps)

# Creating a DataFrame from a dictionary
data = {
    'Crop':,
    'Plot':,
    'Yield': [4.5, 1.8, 3.5, 4.9]
}
df = pd.DataFrame(data)
print("\nPandas DataFrame:\n", df)
```

- **Reading CSV (pd.read_csv):** A fundamental task is loading data. pd.read_csv() is a versatile function to read data from CSV files into a DataFrame.

```
try:
    # Assuming 'crop_data.csv' exists with columns:
Plot,Crop,Yield,Rainfall
    crop_df = pd.read_csv("crop_data.csv")
    print("\nSuccessfully read crop_data.csv")
except FileNotFoundError:
    print("\nError: crop_data.csv not found. Using sample
DataFrame.")
    crop_df = df # Use the previously created DataFrame as a
fallback
```

- **Basic Inspection:** Essential methods to quickly understand your DataFrame :
  - .head(n): View the first n rows (default 5).
  - .tail(n): View the last n rows (default 5).
  - .info(): Get a concise summary (data types, non-null counts, memory usage).
  - .describe(): Calculate summary statistics for numerical columns (count, mean, std, min, max, quartiles).
  - .shape: Get the dimensions (rows, columns) as a tuple.
  - .columns: Get the column labels (names).

```
print("\nInspecting DataFrame:")
print("First 3 rows (.head(3)):\n", crop_df.head(3))
print("\nDataFrame Info (.info()):")
crop_df.info()
print("\nSummary Statistics (.describe()):\n", crop_df.describe())
print(f"\nShape: {crop_df.shape}")
```

```
print(f"Columns: {list(crop_df.columns)}")
```

- **Selecting Columns:** Access columns like dictionary keys or attributes.

```
# Select a single column (returns a Series)
yield_column = crop_df['Yield']
print("\nYield column (Series):\n", yield_column.head())

# Select multiple columns (returns a DataFrame)
crop_plot_df = crop_df[['Crop', 'Plot']]
print("\nCrop and Plot columns (DataFrame):\n",
crop_plot_df.head())
```

- **Selecting Rows (.loc, .iloc):** Powerful methods for row selection :
  - .loc: Selects rows/columns based on **labels** (index names, column names). Slicing with labels is inclusive.
  - .iloc: Selects rows/columns based on **integer position** (0-based index). Slicing with integers is exclusive of the end position.

```
# Assume default integer index for now
print("\nSelecting Rows:")
# Select row with index label 2 using.loc
print("Row at index label 2
(.loc[span_25](start_span)[span_25](end_span)):\n",
crop_df.loc[span_26](start_span)[span_26](end_span))

# Select row at integer position 1 (second row) using.iloc
print("\nRow at integer position 1
(.iloc[span_11](start_span)[span_11](end_span)):\n",
crop_df.iloc[span_12](start_span)[span_12](end_span))

# Select rows with index labels 1 to 3 using.loc (inclusive)
print("\nRows 1 to 3 (.loc[1:3]):\n", crop_df.loc[1:3])

# Select rows at integer positions 0 to 2 using.iloc (exclusive of
3)
print("\nRows 0 to 2 (.iloc[0:3]):\n", crop_df.iloc[0:3])
```

- **Basic Filtering:** Select rows based on conditions (Boolean indexing).

```
# Select rows where Yield > 3.0
high_yield_df = crop_df[crop_df['Yield'] > 3.0]
print("\nHigh Yield Crops (Yield > 3.0):\n", high_yield_df)

# Select rows for 'Maize' crop with Yield > 4.0
high_yield_maize = crop_df[(crop_df['Crop'] == 'Maize') &
(crop_df['Yield'] > 4.0)]
print("\nHigh Yield Maize:\n", high_yield_maize)
```

- **Handling Missing Data:** Real-world data often has missing values (represented as NaN in Pandas).

- .isnull(): Returns a boolean DataFrame indicating True for missing values. Use .isnull().sum() to count missing values per column.
- .dropna(): Removes rows (default) or columns (axis=1) containing missing values.
- .fillna(value): Fills missing values with a specified value (e.g., 0, mean, median).

```
# Introduce a missing value for demonstration
crop_df_nan = crop_df.copy()
crop_df_nan.loc[1, 'Yield'] = np.nan # Set yield for row 1 to NaN
print("\nDataFrame with NaN:\n", crop_df_nan)

print("\nMissing values per column:\n",
crop_df_nan.isnull().sum())

# Drop rows with any NaN values
df_dropped = crop_df_nan.dropna()
print("\nDataFrame after dropping NaN rows:\n", df_dropped)

# Fill NaN values with the mean yield
mean_yield = crop_df_nan['Yield'].mean()
df_filled = crop_df_nan.fillna({'Yield': mean_yield}) # Fill only
in 'Yield' column
print("\nDataFrame after filling NaN with mean yield:\n",
df_filled)
```

- **Adding/Removing Columns:** Modify the DataFrame structure.

```
# Add a new column (e.g., Yield per unit Rainfall - assuming
'Rainfall' column exists)
# crop_df = crop_df['Yield'] / crop_df

# Add a column with a constant value
crop_df = 'Monitored'
print("\nDataFrame with added 'Status' column:\n", crop_df.head())

# Remove the 'Status' column
crop_df_dropped_col = crop_df.drop('Status', axis=1) # axis=1
specifies column
print("\nDataFrame after dropping 'Status' column:\n",
crop_df_dropped_col.head())
# To modify inplace: crop_df.drop('Status', axis=1, inplace=True)
```

- **Basic Grouping (.groupby()):** Powerful for calculating statistics for different groups within the data.

```
# Calculate average yield per crop type
avg_yield_per_crop = crop_df.groupby('Crop')['Yield'].mean()
print("\nAverage Yield per Crop:\n", avg_yield_per_crop)

# Calculate total yield per crop type
total_yield_per_crop = crop_df.groupby('Crop')['Yield'].sum()
print("\nTotal Yield per Crop:\n", total_yield_per_crop)
```

- **Writing to CSV (.to_csv):** Save your processed DataFrame back to a CSV file.

```
try:
    # Save the DataFrame (use df_filled from NaN example)
    df_filled.to_csv("processed_crop_data.csv", index=False) #
index=False avoids writing the DataFrame index
    print("\nProcessed data saved to processed_crop_data.csv")
except IOError:
    print("\nError: Could not write processed data to CSV.")
```

- **Exercises:**
  1. **Load & Inspect:** Load a sample CSV dataset (e.g., student grades, sensor readings). Use .head(), .info(), .describe() to inspect it.
  2. **Selection:** Select specific columns relevant to a task. Select rows based on a condition (e.g., grades above 80, temperatures below freezing). Select data using both .loc and .iloc.
  3. **Missing Data:** Check for missing values. Fill missing numerical values with the column mean. Drop rows where a critical value is missing.
  4. **Aggregation:** Group the data by a categorical column (e.g., 'Subject', 'SensorType') and calculate the average of a numerical column for each group.
  5. **Save:** Save the cleaned and processed DataFrame to a new CSV file..

3.3 Module 7: Visualizing Engineering Data with Matplotlib & Seaborn (Lecture: 2 hrs, Lab: 2 hrs)

Visualizing data is crucial for understanding trends, patterns, outliers, and communicating findings effectively. Matplotlib is the foundational plotting library in Python, while Seaborn builds upon it to create more statistically informative and aesthetically pleasing plots.

- **Introduction to Data Visualization:** Explain why visualizing data (e.g., sensor readings over time, distribution of experimental results) is often more insightful than looking at raw numbers or tables.
- **Basic Plotting with Matplotlib (plt):** matplotlib.pyplot provides a MATLAB-like interface for creating plots. The standard import is import matplotlib.pyplot as plt.
  - plt.plot(x, y): Creates line plots, good for showing trends over time or sequences.
  - plt.scatter(x, y): Creates scatter plots, ideal for visualizing the relationship between two numerical variables.
  - plt.bar(categories, values): Creates bar charts, suitable for comparing quantities across discrete categories.
  - plt.hist(data, bins=...): Creates histograms, showing the distribution of a single numerical variable.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd # Assuming crop_df is loaded from previous
module

# Sample time data
time = np.arange(1, 11) # Days 1 to 10
```

```
temperature = np.array([22, 21, 23, 25, 26, 24, 23, 22, 24, 25]) +
np.random.randn(10)*0.5

plt.figure(figsize=(8, 5)) # Create a figure space

# --- Line Plot ---
plt.plot(time, temperature, marker='o', linestyle='-', color='b')
# Plot time vs temp
plt.title("Daily Average Temperature")
plt.xlabel("Day")
plt.ylabel("Temperature (°C)")
plt.grid(True)
plt.show() # Display the plot

# --- Scatter Plot ---
# Assuming crop_df has 'Rainfall' and 'Yield' columns
# plt.figure(figsize=(8, 5))
# plt.scatter(crop_df, crop_df['Yield'], alpha=0.7)
# plt.title("Yield vs Rainfall")
# plt.xlabel("Rainfall (mm)")
# plt.ylabel("Yield (tons/ha)")
# plt.grid(True)
# plt.show()

# --- Bar Chart ---
# Use the grouped data from Pandas module
avg_yield_per_crop = crop_df.groupby('Crop')['Yield'].mean()
plt.figure(figsize=(7, 5))
plt.bar(avg_yield_per_crop.index, avg_yield_per_crop.values,
color=['cornflowerblue', 'lightcoral', 'lightgreen'])
plt.title("Average Yield per Crop Type")
plt.xlabel("Crop Type")
plt.ylabel("Average Yield (tons/ha)")
plt.show()

# --- Histogram ---
plt.figure(figsize=(8, 5))
plt.hist(crop_df['Yield'], bins=5, color='skyblue',
edgecolor='black') # Visualize yield distribution
plt.title("Distribution of Crop Yields")
plt.xlabel("Yield (tons/ha)")
plt.ylabel("Frequency (Number of Plots)")
plt.grid(axis='y', alpha=0.7)
plt.show()
```

- **Customizing Plots:** Making plots informative requires customization :
    - plt.title("Title"): Adds a title to the plot.
    - plt.xlabel("X-axis Label"), plt.ylabel("Y-axis Label"): Adds labels to the axes.

- plt.legend(): Displays a legend (requires plots to have label arguments, e.g., plt.plot(x, y, label='Data Series')).
- plt.grid(True): Adds a grid.
- plt.figure(figsize=(width, height)): Controls the figure size in inches.
- Color, marker, linestyle arguments within plotting functions (e.g., color='red', marker='x', linestyle='--').

```
# Example with customization and legend
time = np.arange(5)
sensor1_data =
np.array([span_68](start_span)[span_68](end_span)[span_69](start_s
pan)[span_69](end_span)[span_70](start_span)[span_70](end_span)[sp
an_71](start_span)[span_71](end_span)[span_72](start_span)[span_72
](end_span))
sensor2_data = np.array([9, 11, 10, 12, 11])

plt.figure(figsize=(8, 5))
plt.plot(time, sensor1_data, marker='o', linestyle='-', color='r',
label='Sensor 1')
plt.plot(time, sensor2_data, marker='s', linestyle='--',
color='g', label='Sensor 2')

plt.title("Sensor Readings Over Time")
plt.xlabel("Time (hours)")
plt.ylabel("Reading Value")
plt.legend() # Display the legend based on 'label' arguments
plt.grid(True)
plt.show()
```

- **Saving Plots (plt.savefig):** Saves the current figure to a file (e.g., PNG, PDF, JPG). Call this *before* plt.show().

```
# (Code from previous example...)
plt.savefig("sensor_readings.png", dpi=300) # Save as PNG with 300
DPI
plt.show()
```

- **Introduction to Seaborn:** Seaborn simplifies the creation of common statistical plots and often provides more visually appealing defaults. It works seamlessly with Pandas DataFrames. Common import: import seaborn as sns.
  - sns.histplot(data=df, x='column', kde=True): Histogram with optional Kernel Density Estimate.
  - sns.lineplot(data=df, x='x_col', y='y_col'): Line plot, often used with time series.
  - sns.boxplot(data=df, x='category', y='value'): Box plot to show distributions across categories.
  - sns.scatterplot(data=df, x='x_col', y='y_col', hue='category'): Scatter plot with optional coloring by category.

```
import seaborn as sns

# Seaborn Histogram (using crop_df)
```

```
plt.figure(figsize=(8, 5))
sns.histplot(data=crop_df, x='Yield', bins=5, kde=True) # Add
smooth density curve
plt.title("Distribution of Crop Yields (Seaborn)")
plt.xlabel("Yield (tons/ha)")
plt.ylabel("Frequency")
plt.show()
plt.savefig("yield_histogram_seaborn.png")

# Seaborn Box Plot (comparing yield across crops)
plt.figure(figsize=(7, 5))
sns.boxplot(data=crop_df, x='Crop', y='Yield')
plt.title("Yield Distribution by Crop Type (Seaborn)")
plt.xlabel("Crop Type")
plt.ylabel("Yield (tons/ha)")
plt.show()
plt.savefig("yield_boxplot_seaborn.png")
```

- **Using Pandas Plotting (.plot()):** DataFrames and Series have built-in .plot() methods that act as convenient wrappers around Matplotlib for quick visualizations.

```
# Quick plot directly from DataFrame
# Assume crop_df has a 'Date' column and set it as index first
# crop_df_indexed = crop_df.set_index('Date')
# crop_df_indexed['Yield'].plot(figsize=(10, 5), title='Yield Over
Time (Pandas Plot)')
# plt.ylabel("Yield (tons/ha)")
# plt.show()

# Quick histogram from a Series
crop_df['Yield'].plot(kind='hist', bins=5, title='Yield Histogram
(Pandas Plot)')
plt.xlabel("Yield (tons/ha)")
plt.show()
```

- **Exercises:**
  1. **Line Plot:** Plot the temperature data created earlier against time using Matplotlib. Add labels and a title. Save the plot.
  2. **Scatter Plot:** Create a scatter plot of Rainfall vs Yield from the crop_df data using Matplotlib. Customize marker style and color.
  3. **Bar Chart:** Create a bar chart showing the count of plots for each crop type in crop_df using Matplotlib or Seaborn.
  4. **Histogram:** Create a histogram of the Rainfall data using Seaborn. Experiment with the number of bins (bins=...) and add a KDE curve (kde=True).
  5. **Pandas Plot:** Use the Pandas .plot() method to quickly generate a line plot or histogram from a column in your DataFrame..

3.4 Module 8: Introduction to SciPy for Scientific Tasks (Lecture: 1 hr,

Lab: 1 hr)

While NumPy provides the fundamental array structures and basic operations, SciPy builds upon it to offer a vast collection of algorithms for common scientific and engineering tasks. It provides more specialized functions than NumPy.

- **Overview of SciPy's Role:** SciPy contains modules for optimization, linear algebra, integration, interpolation, special functions, signal and image processing, statistics, and more. Think of it as a toolbox filled with specialized instruments for scientific analysis, all designed to work efficiently with NumPy arrays.
- **Key Sub-packages Relevant to Engineering:** Some of the most relevant sub-packages for SUA engineers include :
    - scipy.optimize: Functions for finding minima/maxima of functions, root finding, and curve fitting (e.g., fitting a model to experimental data).
    - scipy.integrate: Tools for numerical integration (calculating definite integrals, e.g., area under a curve) and solving ordinary differential equations (ODEs) (e.g., modeling dynamic systems).
    - scipy.interpolate: Functions for interpolating data (estimating values between known data points, e.g., estimating sensor readings between measurements).
    - scipy.stats: A large collection of statistical functions and probability distributions (e.g., performing t-tests, calculating correlations, working with distributions like normal or binomial).
    - scipy.signal: Tools for signal processing tasks (e.g., filtering noise from sensor data, analyzing frequency components).
    - scipy.linalg: Advanced linear algebra routines, building on NumPy's basic capabilities.
- **Simple Example (Curve Fitting):** A common engineering task is fitting a mathematical model to observed data. scipy.optimize.curve_fit is excellent for this.

```
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

# Example: Fit a quadratic function to some data points
# (e.g., modeling plant height over time, simplified)

# Sample Data (Time in weeks, Height in cm)
time_weeks = np.array()
height_cm = np.array() + np.random.randn(6)*2 # Add some noise

# Define the model function (quadratic: a*t^2 + b*t + c)
def quadratic_model(t, a, b, c):
    return a * t**2 + b * t + c

# Use curve_fit to find the best parameters (a, b, c)
try:
    params, covariance = curve_fit(quadratic_model, time_weeks,
height_cm)
    a_fit, b_fit, c_fit = params
```

```
    print(f"Fitted parameters: a={a_fit:.2f}, b={b_fit:.2f},
c={c_fit:.2f}")

    # Generate fitted curve data
    time_fit = np.linspace(time_weeks.min(), time_weeks.max(),
100)
    height_fit = quadratic_model(time_fit, a_fit, b_fit, c_fit)

    # Plot original data and fitted curve
    plt.figure(figsize=(8, 5))
    plt.scatter(time_weeks, height_cm, label='Observed Data',
color='red')
    plt.plot(time_fit, height_fit, label='Fitted Quadratic Model',
color='blue')
    plt.title("Plant Height Growth Model (Curve Fitting)")
    plt.xlabel("Time (weeks)")
    plt.ylabel("Height (cm)")
    plt.legend()
    plt.grid(True)
    plt.show()
    plt.savefig("curve_fit_example.png")

except RuntimeError:
    print("Curve fitting failed. Could not find optimal
parameters.")
except Exception as e:
    print(f"An error occurred during curve fitting: {e}")
```

This example shows how SciPy can take experimental data and find the parameters of a theoretical model that best describes it, a powerful technique in engineering analysis.

- **Exercises:**
    1. **Curve Fitting:** Given a small dataset (e.g., time vs. concentration, stress vs. strain) and a simple model function (linear or exponential), use scipy.optimize.curve_fit to find the model parameters and plot the results.
    2. **Integration (Conceptual):** Research the scipy.integrate.quad function. Explain in your own words how you might use it to calculate the total volume of water flowing through a pipe over a period if you had a function describing the flow rate over time. (No coding required for this part, just understanding).

Assignment 2: Engineering Data Analysis & Visualization Project

Description: This assignment integrates the skills learned in Modules 5, 6, and 7 (NumPy, Pandas, Matplotlib/Seaborn). You will be provided with a small, semi-realistic dataset relevant to an SUA engineering discipline (instructor will provide one, e.g., irrigation_water_quality.csv, crop_yield_treatments.csv, food_storage_temps.csv).

Tasks:

1. **Load Data:** Load the provided CSV dataset into a Pandas DataFrame.
2. **Inspect & Clean:**

    - Inspect the data using .head(), .info(), .describe().
    - Identify and count any missing values using .isnull().sum().
    - Handle missing values appropriately (e.g., fill with mean/median/mode, or drop rows/columns if justified – explain your choice).
3. **Analyze & Manipulate:**
    - Perform relevant calculations or create new columns if necessary (e.g., calculate water usage efficiency, find temperature differences, categorize treatments). Use Pandas and potentially NumPy for calculations.
    - Calculate summary statistics relevant to the dataset (e.g., average water quality parameters, mean yield per treatment, average temperature fluctuation). Use Pandas .describe() or .groupby() methods.
4. **Visualize:** Create at least two different, meaningful visualizations using Matplotlib and/or Seaborn to illustrate key findings or patterns in the data. Examples:
    - A line plot showing a parameter over time.
    - A histogram showing the distribution of a key measurement.
    - A bar chart comparing average results across different categories/treatments.
    - A scatter plot showing the relationship between two variables.
    - A box plot comparing distributions across categories.
    - Ensure your plots have clear titles, axis labels, and legends if appropriate. Save the plots as image files (e.g., PNG).
5. **Report:** Briefly summarize your findings based on the analysis and visualizations. What do the results suggest?

Relevance: This project simulates a typical data analysis workflow encountered by engineers. It requires applying data loading, cleaning, manipulation, aggregation, and visualization techniques using the core scientific Python libraries to extract insights from data.

Section 4: Conclusion and Next Steps

4.1 Course Recap and Key Takeaways

Over these 25 hours, we have journeyed from the fundamental syntax of Python to its powerful application in engineering data analysis and visualization. We covered:

- **Python Fundamentals:** Variables, data types, operators, control flow (if/else, loops), functions, and basic file handling – the essential building blocks of any program.
- **Data Structures:** Lists and dictionaries for organizing data effectively.
- **NumPy:** The bedrock for numerical computation, enabling efficient array operations.
- **Pandas:** The indispensable tool for loading, cleaning, manipulating, and analyzing tabular data.
- **Matplotlib & Seaborn:** Libraries for creating informative and compelling data visualizations.
- **SciPy Introduction:** A glimpse into the advanced scientific algorithms available for more complex engineering problems.

The key takeaway is that Python is not just a programming language; it is a powerful problem-solving environment particularly well-suited for engineers at SUA. Its readability, extensive libraries, and supportive community make it an invaluable asset for analyzing experimental data, modeling systems, automating tasks, and ultimately, driving innovation in agricultural, irrigation, bioprocessing, and food science engineering. The skills you have begun

to develop here form a strong foundation for tackling complex challenges throughout your academic and professional careers.

4.2 Resources for Further Learning

This course provides a foundation, but the journey of learning Python is ongoing. To continue developing your skills and applying them effectively, consider these resources:

- **Official Documentation:** The official documentation is the ultimate reference guide. It's comprehensive, accurate, and constantly updated.
    - Python:
    - NumPy:
    - Pandas: (Look for the User Guide and API reference)
    - Matplotlib: (Tutorials, examples gallery)
    - SciPy: (Tutorials, reference guide)
- **Online Learning Platforms:** Many platforms offer structured courses, often with interactive exercises.
    - DataCamp: Offers courses specifically on data science topics using Python, including Pandas, NumPy, Matplotlib, and Seaborn.
    - Coursera, edX, Udacity: Host university-level courses on Python programming and data science.
- **Tutorial Websites:** Websites provide numerous tutorials, examples, and practice problems.
    - **GeeksforGeeks**: Extensive collection of articles and exercises on Python and its libraries.
    - Programiz: Clear tutorials and examples for beginners.
    - w3Schools: Clear tutorials and examples for beginners.
    - W3Resource: Large collection of practice exercises with solutions.
    - Real Python: In-depth tutorials on various Python topics.
- **Community & Help:**
    - Stack Overflow: A question-and-answer site where you can find solutions to specific programming problems and ask your own questions. Search thoroughly before asking!
    - GitHub: Explore projects, contribute to open source, and see how others use Python.
- **Apply Your Skills:** The best way to learn is by doing. Apply Python to problems you encounter in your other engineering courses at SUA. Try automating a calculation, analyzing data from a lab experiment, or simulating a simple process related to your field.
- **Explore Further:** As your skills grow, explore more advanced or specialized libraries relevant to your engineering discipline. Examples include:
    - **BioPython:** For bioinformatics and computational biology tasks.
    - **GDAL/Rasterio/GeoPandas:** For working with geospatial data (relevant for irrigation and agricultural mapping).
    - **Scikit-learn:** For machine learning tasks (e.g., predictive modeling).
    - **OpenCV:** For computer vision tasks (e.g., analyzing images of crops or food products).

Continue practicing, exploring, and applying Python. It is a skill that will serve you well throughout your engineering journey here at Sokoine University of Agriculture and beyond. *Kila*

la kheri! (All the best!)

Assignments

Section 1: Introduction to Python for SUA Engineers

1. Why is a programming language like Python relevant for an Agricultural Engineering student focusing on soil science in the Southern Highlands of Tanzania?
2. Consider the challenge of post-harvest losses for mango farmers in the Tanga region. How might Python, as described in the course, be used by a Bioprocessing Engineer to analyze storage conditions and suggest improvements?
3. Imagine you are an Irrigation Engineer working on a project in the Kilombero Valley. Give two specific examples of how Python libraries like Pandas and Matplotlib could help you manage water resources more effectively.
4. A Food Science student at SUA is researching the nutritional content of different cassava varieties grown in the Mwanza area. Explain how Python could assist in organizing and analyzing this data.
5. Beyond specific calculations, what are two advantages of Python's design philosophy (readability, large community) for collaborative engineering projects within Tanzania?
6. Looking at the Course Objectives, which objective will be most immediately helpful in analyzing data from a small-scale farming experiment you might conduct at the Morogoro campus?
7. Explain the difference between the Anaconda distribution and just installing plain Python. Why is Anaconda recommended for engineering students at SUA?
8. If a student is struggling with installing NumPy or Pandas, what resources mentioned in the introduction might they consult first?
9. How does the concept of using libraries like NumPy and Pandas prevent engineers from "reinventing the wheel" when tackling data analysis tasks every day in Tanzanian agriculture?
10. Describe a hypothetical scenario where a Jupyter Notebook could be used to document and share the results of an analysis on cashew nut quality data from Mtwara.

Section 2: Python Programming Fundamentals

1. Write a Python snippet using `input()` to ask a farmer for the area of their *shamba* (farm) in hectares and the expected maize yield in bags per hectare. Then, calculate and `print()` the total expected yield. Remember to use `float()` for conversion.
2. Explain the importance of correct indentation in Python using a simple `if`/`else` example related to checking if the rainfall amount (in mm) recorded in Dodoma is sufficient for planting sorghum (e.g., > 50mm).
3. A Bioprocessing Engineer needs to store the optimal temperature range for fermenting *mtindi* (yogurt). Which Python data type (`int`, `float`, `str`, `bool`) would be most suitable for storing the minimum temperature (e.g., 40.5 °C) and the maximum temperature (e.g., 45.0 °C)? Why?
4. Using arithmetic operators, write a Python expression to calculate the total number of coffee trees needed for a 3-hectare farm in the Kilimanjaro region, assuming a planting density of 1300 trees per hectare.

5. Create a `for` loop that iterates through a list containing the monthly rainfall (mm) for Morogoro: `rainfall_morogoro =`. Inside the loop, use an `if` statement to print only the months with rainfall greater than 100mm.
6. Write a `while` loop that simulates filling a 10,000-litre water tank used for irrigation in Arusha. Assume water flows in at 500 litres per hour. The loop should print the current water level every hour and stop when the tank is full.
7. You have a list of common pests affecting maize in Iringa: `pests =`. How would you use list indexing and slicing to: a) Get the second pest? b) Get the last two pests?
8. Create a dictionary to store the average yield (in tonnes/hectare) for different crops grown in the Rukwa Valley: Maize (3.5), Rice (4.0), Sunflower (1.5). Then, write code to: a) Access the yield of Rice. b) Add Beans with a yield of 1.2 tonnes/hectare.
9. Define a Python function called `calculate_fertilizer` that takes the area of a field (in hectares) and the recommended nitrogen rate (kg/hectare) as input parameters. The function should `return` the total amount of nitrogen fertilizer needed (in kg).
10. Imagine you have a text file named `soil_ph_mbeya.txt`, where each line contains a pH reading from a different farm. Write a Python snippet using `with open(...)` to read each line from the file and print only the readings that indicate acidic soil (pH < 6.0). Handle potential `FileNotFoundError`.

Section 3: Python Libraries for Engineering Applications

1. You have collected daily temperature readings for a greenhouse in Lushoto for a month (30 readings). Why would using a NumPy array be more efficient for calculating the average temperature than using a standard Python list?
2. Create a NumPy array representing the yield (in kg) of 5 tomato plots: `yields = np.array([25.5, 30.1, 22.0, 28.5, 31.2])`. Write NumPy code to calculate: a) The total yield. b) The average yield per plot. c) The yield if each plot increased by 10% (element-wise multiplication).
3. You are given a CSV file, `tanzania_cashew_exports.csv,` with the following columns: 'Year', 'Region', 'Export Volume (Tonnes)', and 'Price (USD per Tonne)'. Write the Pandas code to: a) Read the CSV into a DataFrame called `cashew_df`. b) Display the first 5 rows. c) Show summary statistics for the numerical columns.
4. Using the `cashew_df` DataFrame from the previous question, write Pandas code to: a) Select only the 'Region' and 'Export_Volume_Tonnes' columns. b) Select all rows where the 'Region' is 'Mtwara'. c) Select rows from index 10 to 20 using `.iloc`.
5. Suppose the `cashew_df` has some missing values (`NaN`) in the 'Price_USD_per_Tonne' column. How would you check how many values are missing in that specific column? Describe one method to handle these missing values (e.g., filling with the mean price).
6. Using the `cashew_df`, how would you use the `.groupby()` method in Pandas to calculate the *average* 'Export_Volume_Tonnes' for each 'Region'?
7. You have two NumPy arrays: `time = np.arange(0, 24, 2)` (representing hours) and `temp = np.array([22, 23, 25, 27, 29, 30, 31, 30, 28, 26, 24, 23])` (representing temperature in °C in a Dar es Salaam food storage unit). Write Matplotlib code to create a line plot of temperature vs. time, adding appropriate axis labels and a title.
8. Using the `cashew_df` (assuming it is loaded), write Seaborn code to create a histogram that shows the distribution of 'Price_USD_per_Tonne'. Add a title and axis labels.
9. How would you use Matplotlib or Seaborn to create a bar chart comparing the average

cashew 'Export_Volume_Tonnes' across different regions (using the grouped data from question 6)?
10. A researcher has data on the effect of different fertilizer amounts (x-values) on crop yield (y-values) for a farm in Morogoro. Explain how `scipy.optimize.curve_fit` could be used to fit a mathematical model (e.g., a linear or quadratic function) to this data to understand the relationship.

Section 4: Reflections

1. Reflecting on the course, explain in your own words the most critical reason why Python is a valuable tool for engineers working in the Tanzanian agricultural sector.
2. Which specific Python library (NumPy, Pandas, Matplotlib/Seaborn, SciPy) do you anticipate using most frequently for analyzing data from SUA's agricultural research projects, and why?
3. Imagine you need to automate the process of downloading daily weather data for Mbeya from an online source and saving it to a CSV file. Which Python concepts and libraries covered in this course would be essential for this task?
4. If you encounter a specific error while using Pandas to merge two datasets containing information about livestock in different regions of Tanzania, which resource mentioned in section 4.2 would likely be the most helpful for finding a solution?
5. How can contributing to or exploring open-source Python projects on platforms like GitHub benefit an engineering student at SUA?
6. Beyond the libraries covered, the course mentioned `GDAL/Rasterio/GeoPandas`. How might these libraries be specifically valuable for an Irrigation and Water Resources Engineer working on mapping water canals near Lake Victoria?
7. The course mentioned `Scikit-learn` for machine learning. Describe a simple predictive task relevant to Tanzanian agriculture (e.g., predicting crop disease likelihood based on sensor data) where this library could be applied.
8. How does the principle of writing functions to create reusable code blocks help in developing larger, more complex analysis scripts for long-term agricultural monitoring projects in Tanzania?
9. Considering the data challenges in Tanzania (e.g., potentially incomplete or inconsistent data collection), why are the data cleaning techniques learned in Pandas (handling missing values, duplicates) critical?
10. What is the first step you would take *after* this course to apply your Python skills to a real problem or dataset related to your specific engineering field at SUA?

Combined Sections Assignment

1. **Multi-Crop Profitability Calculator:**

- **Scenario:** Farmers need to decide which crops are most profitable based on varying input costs and market prices.
- **Task:** Create a program that reads crop data (expected yield per acre, seed cost, fertilizer cost, pesticide cost, labor cost per acre) from a data file. It should also read current market prices per kg/unit for various crops from another file. The program must allow the user to input the acreage for different crops they plan to plant and calculate the estimated total cost, total revenue, and net profit *per crop* and *overall*, storing and

retrieving the necessary cost and price data effectively.

2. **Custom Fertilizer Blend Recommender:**

- **Scenario:** Different crops have varying nutrient requirements (N, P, K), and soil tests reveal deficiencies. Farmers buy basic fertilizer components.
- **Task:** Develop a tool that takes soil test results (levels of N, P, K) and the target crop as input. Use stored data mapping crops to their ideal nutrient ratios and another dataset listing available fertilizer components with their N-P-K content and price per kg. Recommend a cost-effective blend of available components to meet the crop's needs for a given area, calculating the total cost using the stored data.

3. **Small-Scale Food Processor Batch Tracking & Costing:**

- **Scenario:** A small food processor (e.g., making chili sauce, drying mangoes) needs to track ingredients used per batch, calculate production costs, and assign batch numbers for traceability.
- **Task:** Build a system where the user can manage recipes (ingredients and quantities) stored persistently. When starting a new batch, the user enters the recipe name and batch size. The system reads ingredient costs from a stored data file. It must calculate the total cost for the batch, generate a unique batch ID, and log the batch details (ID, date, recipe, size, ingredient list, total cost) to a persistent log file.

4. **Localized Pest/Disease Identification Aid:**

- **Scenario:** Farmers observe symptoms on their crops but aren't sure about the cause or treatment.
- **Task:** Create a program that stores information about common pests and diseases affecting major Tanzanian crops (symptoms, affected crops, description, control methods). This data should be stored in a structured file. The user inputs the crop and observed symptoms. The program must search the stored data and suggest possible pests/diseases matching the symptoms for that crop, along with control recommendations.

5. **Farm Cooperative Member Management System:**

- **Scenario:** Agricultural cooperatives need to manage member details, track their produce contributions, and calculate payouts.
- **Task:** Develop a system that stores member information (ID, name, contact, crops grown) persistently. Create a way to log produce deliveries (date, member ID, crop type, quantity, quality grade) into a data file. The system should allow adding/updating members, recording contributions, and generating reports like total contribution per member per crop, calculating payouts based on grade and pre-set prices loaded from storage.

6. **Irrigation Schedule Advisor:**

- **Scenario:** Optimizing water usage is crucial. Farmers need guidance on when and how much to irrigate based on crop, soil, and weather.
- **Task:** Create a tool that uses stored data defining water needs for different crops at various growth stages and factors for soil types affecting water retention. The user inputs the crop, current growth stage, soil type, and date/amount of last rainfall/irrigation (which should be read from and written to a log file). The program estimates the current soil

moisture deficit based on stored crop and soil data, as well as recent water events, and advises whether irrigation is needed, suggesting the appropriate amount.

7. **Livestock Feed Formulation Calculator:**

- **Scenario:** Farmers mixing their own livestock feed need to meet nutritional requirements cost-effectively.
- **Task:** Develop a program that reads available feed ingredients (e.g., maize bran, sunflower cake) and their nutritional profiles and costs per kg from a data file. Store target nutritional requirements for different animal types/stages in another data structure. The user selects the animal type and desired batch weight. The program must calculate a feed mix using selected ingredients from the stored data to approximate the target nutritional profile, showing the total cost and nutritional breakdown.

8. **Harvest Yield Forecaster:**

- **Scenario:** Farmers want an early estimate of their potential harvest yield based on planting data and observations.
- **Task:** Create a tool where users log planting details (crop, date, area, variety) into a data file. Use stored data containing average yield potential for different varieties and factors affecting yield (e.g., rainfall score, pest severity score). The user periodically updates the status with these scores. The program uses these factors and the stored base yield data to adjust and provide a forecasted yield range, saving the planting log persistently.

9. **Local Market Price Aggregator & Analyzer:**

- **Scenario:** Farmers need to know current prices in different local markets to decide where to sell.
- **Task:** Build a program where users can record daily prices for various crops from different Tanzanian markets. This data (date, market, crop, unit, price TZS) must be stored persistently in a data file. The program should allow users to query the stored data for prices of a specific crop across markets, view price history for a crop in one market, and calculate average prices or identify market price variations.

10. **Food Processing Quality Control Logger:**

- **Scenario:** Food processors need to document quality control checks for compliance and internal monitoring.
- **Task:** Create a system to log QC checks. Define standard check parameters per product type using stored data structures. When a check is performed, the user selects the product, enters the batch ID, and records the results for each parameter according to the stored protocol. Log the complete QC data (timestamp, product, batch ID, results, operator) to a persistent data file.

11. **Simple Farm Asset Inventory Manager:**

- **Scenario:** Farmers need to keep track of their tools, equipment, and major assets.
- **Task:** Develop a program to manage a farm asset inventory. Store asset details (ID, name, category, purchase date, price, status) in a structured, persistent file. Allow adding, updating (especially status), removing, and viewing assets from the stored data. Implement search/filter functionality based on the stored asset details.

12. **Crop Rotation Planner:**

- **Scenario:** Planning crop rotations helps manage soil health and pests.
- **Task:** Create a tool where a user defines their fields. Store crop family information and rules about good/bad rotation sequences using persistent data structures. The user records the planting history for each field (year, season, field name, crop) in a data file. The tool must analyze the stored history for a chosen field and suggest suitable crops for the next season based on the stored rotation rules.

13. **Poultry Farm Egg/Meat Production Tracker:**

- **Scenario:** Poultry farmers need to monitor egg production or broiler growth rates and feed consumption.
- **Task:** Design a program to track flock performance. Store flock details persistently. Log daily egg collection, periodic broiler weight checks, and feed consumption into separate data files. The program must calculate key metrics like eggs per hen per day or feed conversion ratio using the data logged in the files.

14. **Seed Viability & Inventory Tracker:**

- **Scenario:** Farmers or seed suppliers need to track seed batches, their source, germination test results, and quantity on hand.
- **Task:** Create a system to manage seed inventory. Store seed batch information (ID, crop, variety, source, date, quantity, location) persistently. Add functionality to record germination test results associated with each batch in the stored data. Allow users to update quantities in the stored inventory file when seeds are used or acquired. Implement querying based on the stored data.

15. **Swahili Agricultural Glossary Tool:**

- **Scenario:** Students, extension workers, or farmers might need quick translations or definitions of agricultural terms between English and Swahili.
- **Task:** Build a simple glossary tool. Store agricultural terms and their Swahili translations/definitions in a persistent data file (e.g., JSON or CSV). The program should allow the user to enter a term in either English or Swahili and retrieve its corresponding translation and definition from the stored data.

16. **Farm Labor Management & Payroll Calculator:**

- **Scenario:** Farmers employing casual or permanent laborers need an efficient way to track work hours or days and calculate wages accurately, especially during peak seasons like planting or harvesting in regions like Morogoro.
- **Task:** Develop a system to record worker details (e.g., Name, National ID/Reference, Contact, Pay Rate in TZS per day or per hour) stored persistently. Allow logging daily work records (Date, Worker ID/Name, Task Description, Hours/Days worked) into a data file. The program must be able to calculate wages due for a specific period (e.g., weekly) per worker by retrieving their rate and logged work details from the stored files.

17. **Direct Customer Order Tracker (Farm-to-Consumer):**

- **Scenario:** Smallholder farmers or food processors in Tanzania who are beginning to sell directly to local consumers, hotels, or restaurants need a simple system to manage these orders.
- **Task:** Create a program to manage customer orders. Store customer information (Name, Contact, Delivery Location - e.g., area in Morogoro town) and a list of available products

with prices (e.g., 'Fresh Milk litre', 'Organic Tomato kg', 'Dried Mango packet') persistently. Allow users to record new orders (Customer Name, Order Date, Product(s), Quantities, Total Price TZS, Order Status [e.g., Pending, Out for Delivery, Delivered, Paid]) into an order log file. The program should allow viewing open/pending orders and updating their status, retrieving customer and product details from stored data.

18. **Post-Harvest Loss Tracking Tool:**

- **Scenario:** Reducing post-harvest losses is critical for food security and farmer income. Farmers need a way to identify where and why losses occur in crops like maize, tomatoes, or fruits.
- **Task:** Build a tool to track post-harvest losses systematically. Define common stages where losses occur (e.g., Harvesting, Field Transport, Storage, Market Transport) using a stored configuration. Allow users to log loss incidents by recording the Date, Crop Type, Initial Quantity (e.g., in kgs or *debe*), Stage of Loss, Estimated Quantity Lost, and suspected Reason (e.g., Pests, Spoilage, Physical Damage) into a data file. The program must read this log file and provide summaries of losses by crop, stage, or reason over time.

19. **Farm Machinery Usage & Maintenance Log:**

- **Scenario:** Farms that utilize shared or owned machinery, such as tractors, water pumps, or small processing equipment (e.g., a maize sheller), need to track usage time, fuel consumption, and upcoming maintenance needs.
- **Task:** Develop a digital logbook system for farm machinery. Store essential details for each machine (e.g., Machine ID, Type [Tractor, Pump], Model/Name, Purchase Date) persistently. Allow users to log usage entries (Date, Machine ID, Task Performed, Hours Used, Fuel Added [Litres], Operator Name) and maintenance records (Date, Machine ID, Maintenance Description, Parts Used, Cost TZS) into separate data files. The program must allow users to retrieve and view the usage history or maintenance logs for specific machinery from the stored files.

20. **Organic Farming Input & Practice Logger:**

- **Scenario:** Farmers transitioning to or practicing organic farming need to keep detailed records of allowed inputs and field activities to meet certification standards or simply manage their system effectively.
- **Task:** Create a digital logbook specifically for organic farming practices. Store a list of approved organic inputs (e.g., specific compost types, bio-pesticides, certified organic seeds) with their details persistently. Allow users to log activities associated with particular fields or plots (Date, Field ID/Name, Activity Type [e.g., Planting, Manure Application, Cover Cropping, Pest Scouting, Weeding], Specific Input Used [if any] from the stored list, Application Rate/Quantity, Notes) into a detailed activity log file. The program must allow filtering and viewing the activity log for a specific field, date range, or activity type based on the stored data.

21. **Compost Management Tracker:**

- **Scenario:** Farmers in the Morogoro region making their own compost need to manage the process effectively to ensure good quality fertilizer. This involves tracking ingredients, turning schedules, and potentially temperature to optimize decomposition.
- **Task:** Create a tool to help manage individual compost piles. Allow users to define a

compost pile with a unique ID, start date, and location. Log entries for adding materials (Date, Pile ID, Material Type [e.g., kitchen scraps, manure, maize stalks, green leaves], Estimated Quantity) and management actions (Date, Pile ID, Action [e.g., Turned Pile, Added Water], Temperature Reading [optional], Notes) into a persistent data file. The system should retrieve the history for a specific pile and potentially alert the user when a pile is due for turning based on predefined intervals stored perhaps in a configuration dictionary.

22. **Local Agro-Input Supplier Directory:**

- **Scenario:** Farmers and agricultural students in Morogoro often need quick access to information about local shops or suppliers for essential inputs like seeds, fertilizers, tools, or animal feed.
- **Task:** Develop a simple directory program focused on the Morogoro area. Store supplier details (Supplier Name, Location Description/Ward, Phone Number, Main Products Offered [list/string, e.g., "Seeds, Fertilizer", "Animal Feed", "Farm Tools"], User Notes/Rating) in a structured data file (like JSON or CSV). The program must allow users to add new supplier entries, search for suppliers based on the type of product needed or location, and display the contact details retrieved from the stored file.

23. **Manual Rainfall Data Logger & Analyzer:**

- **Scenario:** Consistent rainfall data is vital for farming decisions, but automated weather stations aren't always accessible. Many farmers rely on simple rain gauges and manual recording.
- **Task:** Create a program designed for manually logging daily rainfall measurements. The user should be able to input the date and the rainfall amount (in mm) recorded for that day. This data must be stored chronologically in a persistent log file (e.g., `rainfall_log.csv`). The program should retrieve data from this log file to calculate and display basic analyses, such as total rainfall for the past 7 days and the current month. It may also compare the current month's total to the same month in previous years, if the log file contains historical data.

24. **Processing Equipment Efficiency Calculator:**

- **Scenario:** Small-scale food processors in Tanzania, for example those pressing sunflower oil or milling maize in Morogoro, need to understand the efficiency of their equipment – how much finished product they get from a certain amount of raw material.
- **Task:** Develop a tool to calculate and track processing efficiency for different types of equipment. Store basic profiles for processes (e.g., 'Sunflower Oil Pressing', 'Maize Milling'). Allow users to log processing batches by entering: Date, Process Type, Raw Material Input (Type and Quantity in kg), and Finished Product Output (Type and Quantity in kg or litres). This log data must be saved to a file. The program must calculate the efficiency percentage (Output Quantity / Input Quantity * 100) for each logged batch and allow the user to view historical efficiency trends for a specific process type by retrieving data from the log file.

25. **Community Seed Exchange Logger:**

- **Scenario:** Farmer groups or local community seed banks often facilitate the saving, storage, and sharing of diverse local seed varieties, which requires careful tracking.
- **Task:** Create a system to manage a community seed exchange inventory. Store

information about seed batches available within the group (unique Seed ID, Crop Type, Variety Name, Year Saved, Contributing Member's Name/ID, Quantity Available [e.g., packets, grams, kg], Storage Location, Brief Notes) in a persistent inventory file. Implement functionality to log transactions when a member 'deposits' (adds) seeds or 'withdraws' (takes) seeds (Transaction Date, Member Name/ID, Seed ID, Quantity Deposited, Quantity Withdrawn), updating the available quantity in the main inventory file. The program should allow searching the inventory file for specific seeds and viewing the transaction history.